

W03 - Debugging and Profiling

Mohammad Amin GHAVAMI

Uni.lu HPC username: "mghavami"

GitHub username: "AminGhavami"

University of Luxembourg

March 18, 2025

1 Question 2:

In your GitHub Class repository, add the following files to the **perfreport** directory:

- The adapted submission script that runs the performance analysis of the motorbike simulation on 6 cores.
Done!
- The two performance reports (**.txt** and **.html**) of the performance analysis.
Done!

Looking at the report, can you tell:

- What is the main bottleneck for the performance of this application? Percentage spent?
"simpleFoam is Compute-bound in this configuration". 81.2% CPU time (72.6% of that, related to "Memory accesses")
- What is the second factor limiting the performance? Percentage spent?
MPI with 18.7% was the second factor
- How much (in percentage) represents the rest? What does it correspond to?
0.2% time spent in filesystem I/O.

Thinking in terms of roofline analysis,

- Is this application limited by the compute operations or the memory accesses?
It's limited by the Memory accesses (with 72.6% of CPU time) based on the report.
- Would this execution benefit from using more vectorized instructions?
Only 7.1% of CPU time, related to the Vector numeric operations. It seems the Scalar numeric is more than it.

Based on this report,

- What is your main recommendation to improve the performance of this simulation?
 - **Physical core utilization** is **low**. Try increasing the number of threads or processes to improve performance.
 - The **peak node memory usage** is **very low**. Running with fewer MPI processes and more data on each process may be more efficient.
 - The per-core performance is **memory-bound**. Use a profiler to identify time-consuming loops and check their cache performance.

2 Question 3:

In your GitHub Class repository, add the following files to the **perfreport** directory:

- The adapted submission script that runs the performance analysis of the motorbike simulation on 24 cores.
Done!
- The two performance reports (.txt and .html) of the performance analysis.
Done!

Looking at the new report,

- Describe in a few lines the main changes in the results, compared to the execution on 6 cores.
The percentage of computing has changed from 81.2 to **54.8**.
- What is the change in the execution time? What is the speedup?
The total time was decreased to 70 seconds (from 162 seconds with 6 cores). Since the simpleFoam is **Compute-bound**, using more cores ($162 \div 70 = 2.3 \text{ speedup the operation}$).
- Is there a significant change in the time spent in I/O?
No, in both cases is around 3.5

3 Question 4:

In your opinion, what are the advantages of using non-interactive profiling over directly opening the graphical interface and setting it up manually?

The main advantage of non-interactive profiling is automation and remote execution in my opinion. Also, reduces performance overhead, supports , and provides reliable, reproducible results are the other advantages. While GUIs are useful for visualization, they are not practical for large-scale profiling in HPC environments.

4 Question 5:

In your GitHub Class repository, add the following files to the **profiling** directory:

- The adapted submission script that runs the profiling of the motorbike simulation on 6 cores.
Done!
- The result trace (file **simpleFoam_XXX.map**) of the performance profiling.
Done!

5 Question 6:

In the MAP user interface,

- Select the metrics preset **CPU Instructions**:
 - What is the type of CPU instructions that occupy the CPU most of the time?
Memory access
 - Take a screenshot.

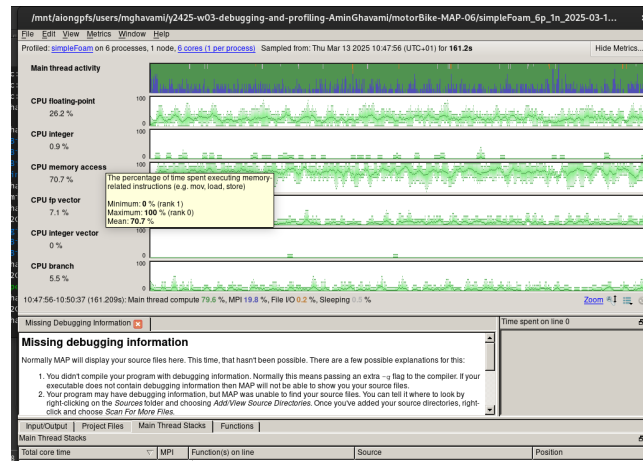


Figure 1: results related to question

- Select the metrics preset **MPI**:
 - What is the average amount of data sent by MPI over the whole simulation?
19.4%
 - If you select the first seconds of the simulation, how does this value change?
it's 3.45%
 - Take a screenshot.

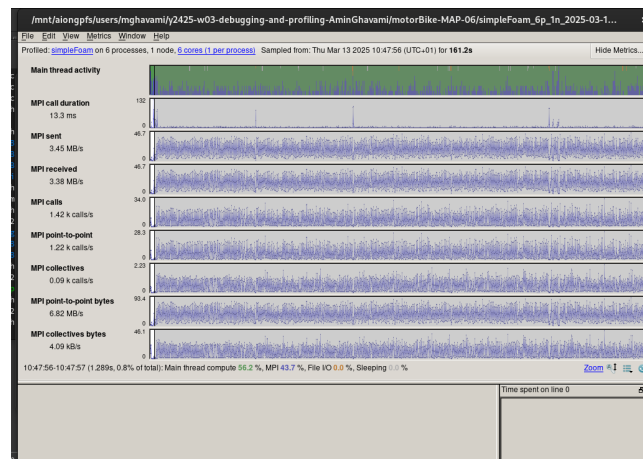


Figure 2: results related to question

- Select the metrics preset **IO**:
 - Can you identify how many times the OpenFOAM output is written to disk during the whole simulation?
6 times
 - Does this match with the files present in the execution directory?
It seems yes
 - Take a screenshot.

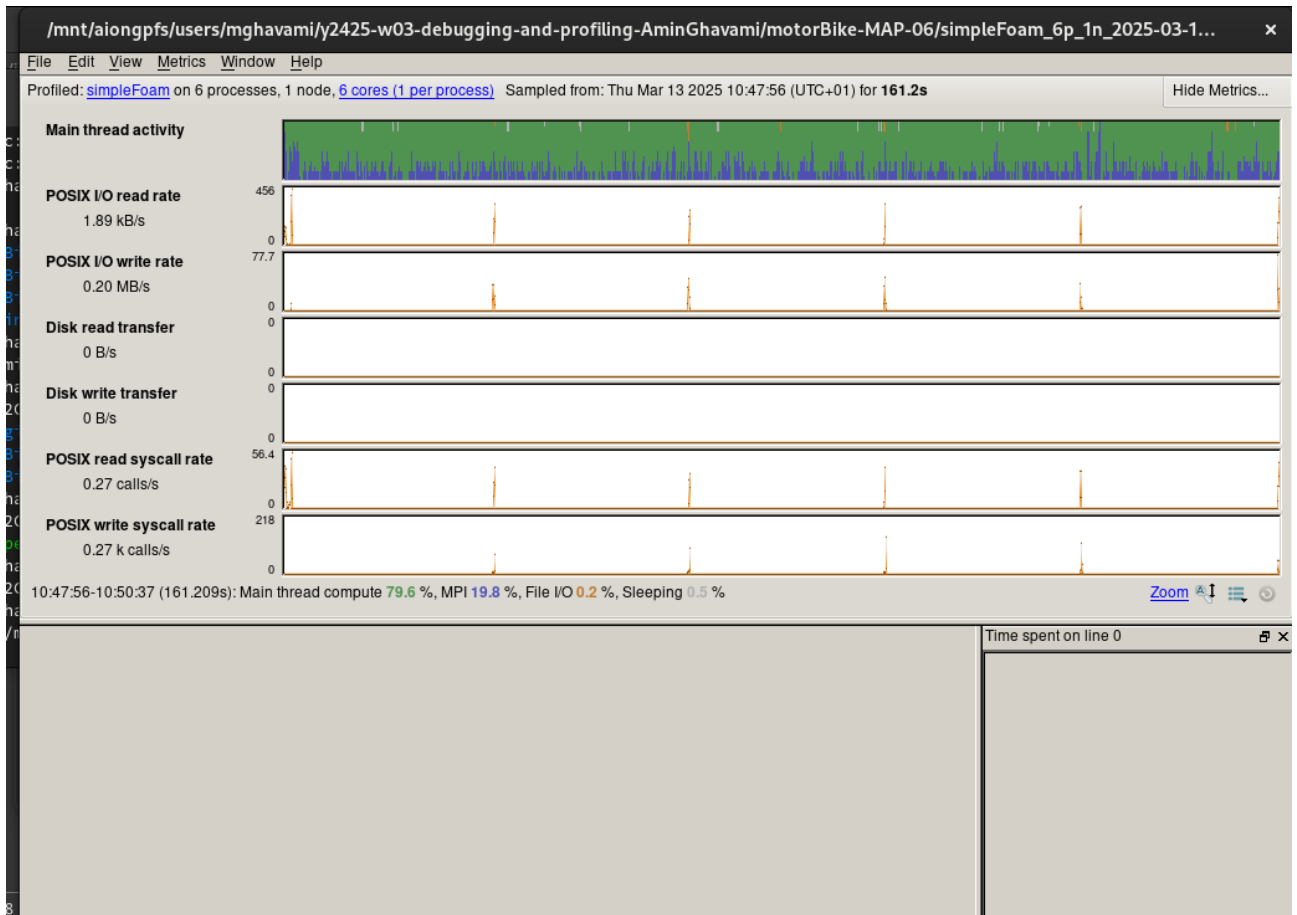


Figure 3: Answers to the question 6

6 Question 7:

In the bottom panel of the MAP user interface, select the tab **Main Thread Stacks**:

- In which function is done most of the MPI communication? (remember the color code)
MPI_Waitall, Source: Foam::UPstream::waitRequests(int)
- Save a screenshot in your report.

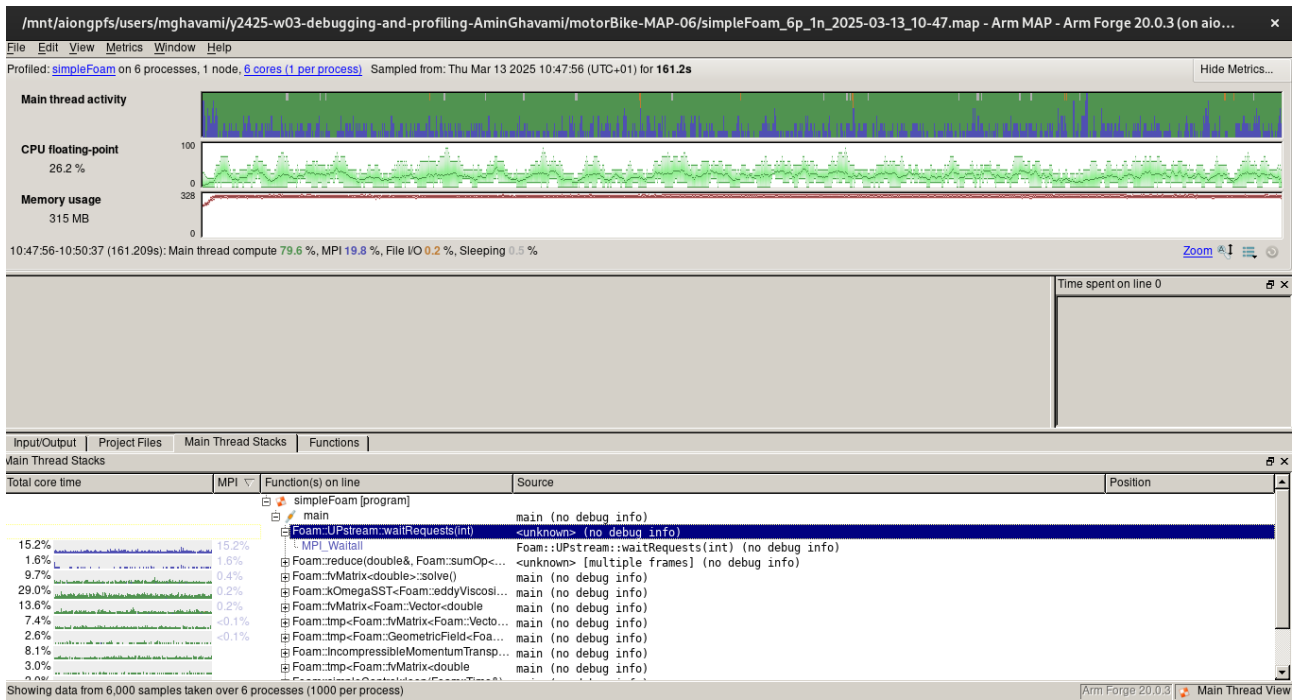


Figure 4: Answers to the question

7 Question 8:

Find the two bugs in the Pi Monte Carlo program. For each bug,

- Explain the cause of the bug.
The first bug is related to the distribution of iterations between cores. The problem appeared because the total number of iterations was not divisible by 3.
The second problem was related to "Integer Overflow" related to this part of code:
`MPI_Reduce(in_circle_count_local, in_circle_count_global, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD)`
- Propose a correction.
for the first bug, I had no idea on C programming language, so tanks to the ChatGPT, I added three lines to the code. But for the second bug, I've changed several variables to avoid integer overflow and finally, I came up with changing `MPI_INT` to `MPI_LONG_LONG_INT`
- Show the results for the correct execution.

```
0 [mghavami@aion-0028 debugging](6843967 1N/32T/32CN)$ make tests
mpirun -n 1 ./pi_monte_carlo 100000000
Nb Processors = 1 - Elapsed Time = 2.575 s - Nb Iterations = 100000000 - Pi = 3.141795600000000022
mpirun -n 2 ./pi_monte_carlo 100000000
Nb Processors = 2 - Elapsed Time = 1.338 s - Nb Iterations = 100000000 - Pi = 3.1418574000000000133
mpirun -n 3 ./pi_monte_carlo 100000000
Nb Processors = 3 - Elapsed Time = 0.893 s - Nb Iterations = 100000000 - Pi = 3.141501519999999825
mpirun -n 4 ./pi_monte_carlo 100000000
Nb Processors = 4 - Elapsed Time = 0.674 s - Nb Iterations = 100000000 - Pi = 3.1415089200000000149
mpirun -n 8 ./pi_monte_carlo 100000000
Nb Processors = 8 - Elapsed Time = 0.336 s - Nb Iterations = 100000000 - Pi = 3.141457799999999967
mpirun -n 16 ./pi_monte_carlo 1000000000
Nb Processors = 16 - Elapsed Time = 16.741 s - Nb Iterations = 1000000000 - Pi = 3.1415725636000000034
```

Figure 5: Answers to the question

Write down these answers in your report. And finally, add the corrected program to your GitHub repository. Done!